Tail-GAN

Learning to Simulate Tail Risk Scenarios

Chao Zhang
HKUST(GZ)

with Rama Cont (Oxford), Mihai Cucuringu (UCLA), Renyuan Xu (NYU)

March 17, 2026

Motivation and Introduction

Score Functions for Risk Measures

Tail-GAN: Design and Theoretical Guarantee

Numerical Performance

TAiLgan
Learning to simulate tail Risk scenarios

Market simulator

What is a market simulator?

- Denote $\mathbb{P}_r$ as the target distribution of some price dynamics over a given time period. Given some samples $\{\rho_i\}_{i=1}^N$ from $\mathbb{P}_r$, the **goal** is to construct $\mathbb{P}_G$ such that

$$\mathbb{P}_G \text{ is "similar" to } \mathbb{P}_r \text{ under certain criterion}$$

Market simulator

What is a market simulator?

- Denote $\mathbb{P}_r$ as the target distribution of some price dynamics over a given time period. Given samples $\{p_i\}_{i=1}^N$ from $\mathbb{P}_r$, the **goal** is to construct $\mathbb{P}_G$ such that

$$\mathbb{P}_G \text{ is "similar" to } \mathbb{P}_r \text{ under some criterion.}$$

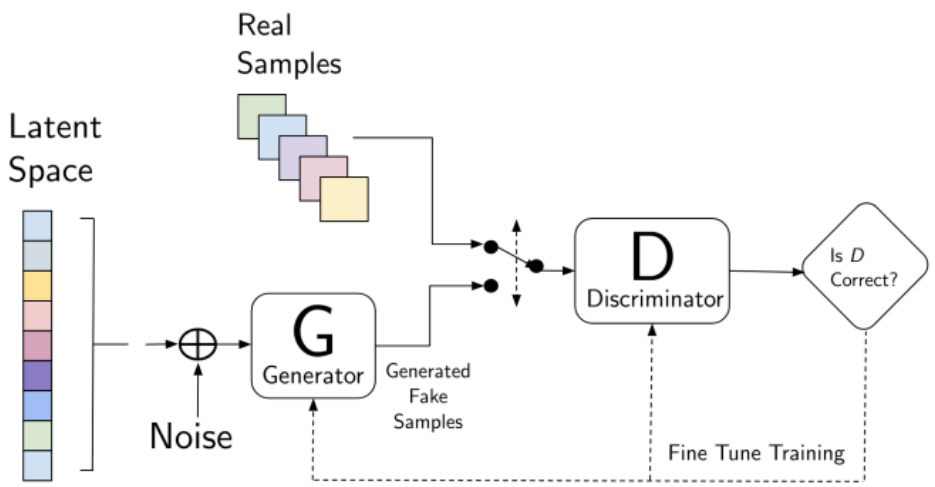
Why do we need a market simulator?

- Key ingredient of quantitative risk management systems.
- Main application: estimate the loss distributions for various trading strategies and portfolios.
- One focus is the calculation of **risk measures**, such as Value-at-Risk and Expected Shortfall.
- Other applications:
  - back-testing and stress testing; improve risk management; portfolio construction; forecasting
  - data protection

Classic simulation models in mathematical finance

- **Parametric approach**: time series models such as GARCH models, diffusion and jump processes, stochastic volatility models, and copulas
  - **Pros:** successful in low-dimensional applications involving a small number of homogeneous assets
  - **Cons:** difficult to model the joint dynamics of large or heterogeneous portfolios with assets from different classes $\Rightarrow$ lack of scalability + risk of model misspecification
- **Gaussian models** ("Black–Scholes models")
  - Default approach in most high-dimensional applications
  - **Cons:** mismatch with stylized facts observed from financial data (e.g., heavy tails, volatility clustering)

Generative Adversarial Networks (GAN)



Goodfellow et al. [2014]

- Robust network used for unsupervised machine learning to build a min-max game between two players: D (the discriminator) and G (the generator)
- Player G tries to fool player D by producing very "natural looking" data from random latent vector z
- Player D gets better at distinguishing between real and generated data

GAN in finance...

- GANs have been used for building 'market simulators in finance(e.g.Koshiyama et al. [2020], Wiese et al. [2020], Buehler et al. [2020], Yoon et al. [2019])by training them onraw returns.
- Training criterion is cross-entropy or some probability metrics, e.g., Wasserstein distance.
- Final result is a 'black-box' simulator that generates market scenarios.

Reference	Model category	Architecture	Loss Function	Data
Quant-GAN [Wiese et al., 2020]	GAN	TCN	KL divergence	Returns
[Buehler et al., 2020]	VAE	NN	l_2 norm	Signatures of paths
Time-GAN [Yoon et al., 2019]	GAN + Supervised Learning	RNN	Combination of log-likelihood and l_2 norm	Time Series
[Fu et al., 2019, Koshiyama et al., 2020]	CGAN	NN	KL divergence	Returns
SigCWGAN [Ni et al., 2020]	CGAN	Residual NN	l_2 norm	Signatures
[Li et al., 2020]	CGAN	LSTM	KL divergence	Limit order books
PCFGAN [Ni et al 2023]	GAN	NN	Characteristic fn	Time series
ExGAN [Bhatia et al., 2020]	CGAN	CNN	KL divergence + Supervised loss	Rainfall images

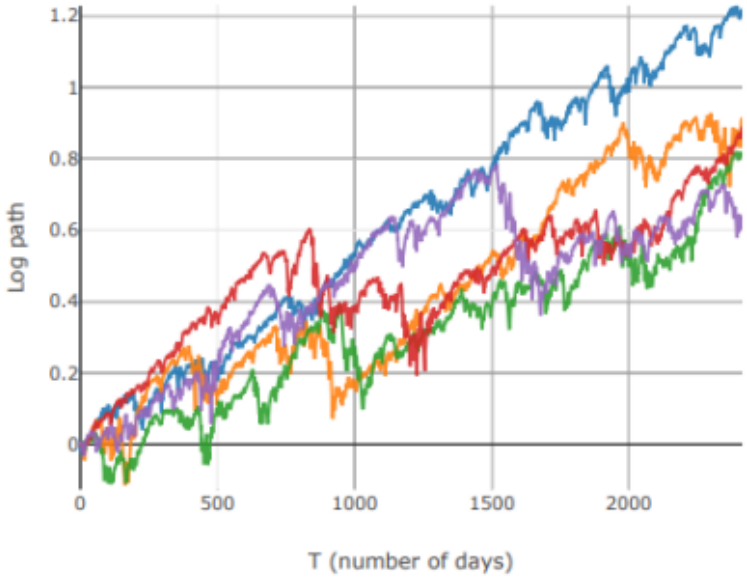
Figure: Table 1: Literature review.

Key questions in model validation

How do we know if the generated scenarios are realistic?

- In the context of images or text, user perception can be easily used for validation
- virtually check is impossible to evaluate the simulated finance scenarios

(a) Image generation of dogs



(b) Financial scenarios: which path is a possible realization of S&P500?

Validation Criterion

Criterion should be tailored to the down-stream tasks of interest.

Prior works focus on "global properties" (such as Wasserstein distance) of the return distribution:

- Global property v.s. Tail behaviour (rare events)?
- Raw return distribution v.s. PnL distribution of trading strategies?

Overview of TAIL-GAN framework

Tail risk and Elicitability:

- Value at Risk (VaR) and Expected Shortfall (ES) are commonly used statistics for measuring the tail risk of a variety of trading strategies
- VaR and ES of a group of trading strategies are incorporated in the loss function by utilizing the joint elicitability property

Discrimination via dynamic trading strategies:

- static portfolios are used to capture cross-asset dependence information
- dynamic strategies are used to capture temporal dependence information in the financial scenarios

Motivation and Introduction

Score Functions for Risk Measures

Tail-GAN: Design and Theoretical Guarantee

Numerical Performance

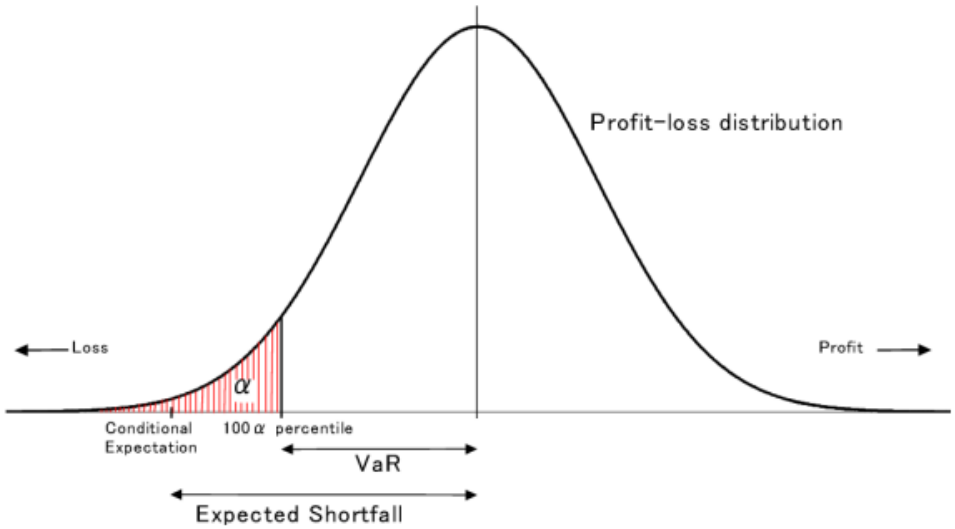
Tail risk measures

Value-at-Risk (VaR) and Expected Shortfall (ES)

Given a one-dimensional distribution μ with a finite mean and a confidence level $0 < \alpha < 1$, the α-VaR and α-ES are defined as

$$\mathrm{VaR}_\alpha(\mu) := \inf\{\, x \in \mathbb{R} : \mu(X \le x) \ge \alpha \,\}$$

$$\mathrm{ES}_\alpha(\mu) := \frac{1}{\alpha} \int_0^\alpha \mathrm{VaR}_\beta(\mu)\, \mathrm{d}\beta$$



Elicitability: from statistical estimation to optimization

Elicitability

A statistical functional T (e.g., mean) is **elicitable** for a set of distributions $\mathcal{F}$ if there is a score function $S(x, y)$ such that

$$T(\mu) = \arg\min_x \int S(x, y)\, \mu(dy)$$

for any distribution $\mu \in \mathcal{F}$.

Examples:

- mean with $S(x, y) = (x - y)^2$
- median with $S(x, y) = |x - y|$
- ES_α is not elicitable but (VaR_α, ES_α) is jointly elicitable
- Spectral risk measure $\Rightarrow$ allows us to incorporate multiple quantile levels

Motivation and Introduction

Score Functions for Risk Measures

Tail-GAN: Design and Theoretical Guarantee

Numerical Performance

Some notations

- Consider the price dynamics of M assets over a time horizon T.
- The user specifies K trading strategies which may be static or dynamic.
- $\Pi^k : \mathbb{R}^{M \times T} \rightarrow \mathbb{R}$ represents the k-th strategy, with the price scenarios $\mathbf{p} \in \mathbb{R}^{M \times T}$ as the input and the final PnL as the output.
- $\Pi^k \# \mathbb{P}_r$ denotes the distribution of the PnL of the k-th strategy $\Pi^k(\mathbf{p})$, where $\mathbf{p} \sim \mathbb{P}_r$.

Theoretical version of the discriminator

- $D : \mathcal{P}(\mathbb{R}) \to \mathbb{R}^2$ a mapping parameterized by a neural network, with strategy PnLs as the input and two values as the output
- **Goal:**

$$\overbrace{D\Bigg(\underbrace{\Pi^k \# \mathbb{P}_r}_{\text{strategy PnL distribution}}\Bigg)}^{\text{VaR and ES prediction from D}} \approx \big(\mathrm{VaR}_\alpha(\Pi^k \# \mathbb{P}_r),\, ES_\alpha(\Pi^k \# \mathbb{P}_r)\big)$$

- **Mathematically,**

$$D^* \in \arg\min_D \frac{1}{K} \sum_{k=1}^{K} \mathbb{E}_{p \sim \mathbb{P}_r}\Big[\, S_\alpha\big(D(\Pi^k \# \mathbb{P}_r); \mathbf{p}\big) \,\Big]$$

The generator

- Input: random noise $\mathbf{z} \sim \mathbb{P}_z$ distribution on the latent space
- Output $G(\mathbf{z})$: simulated price scenario
- Mapping parameterized by neural network:

$$G(\mathbf{z}; \mathbf{W}, \mathbf{b}) = \mathbf{W}_L \, \sigma\Big(\mathbf{W}_{L-1} \cdots \sigma(\mathbf{W}_1 \mathbf{z} + \mathbf{b}_1)\Big) + \mathbf{b}_L,$$

where

- $\mathbf{W} = (\mathbf{W}_1, \ldots, \mathbf{W}_L)$ and $\mathbf{b} = (\mathbf{b}_1, \ldots, \mathbf{b}_L)$ represent the trainable parameters, with $\mathbf{W}_\ell \in \mathbb{R}^{n_\ell \times n_{\ell-1}}$ and $\mathbf{b}_\ell \in \mathbb{R}^{n_\ell \times 1}$ for $\ell = 1, 2, \ldots, L$.
- The operator $\sigma(\cdot)$ is a component-wise non-linear operator.

Objective of the generator

The goal of the generator is to best match the tail risks across the set of K benchmark strategies:

$$\underbrace{D(\Pi^k \# \mathbb{P}_r)}_{\text{strategy PnL distribution}} \approx \Big(\mathrm{VaR}_\alpha(\Pi^k \# \mathbb{P}_r),\ ES_\alpha(\Pi^k \# \mathbb{P}_r)\Big)$$

Objective of the generator

The goal of the generator is to best match the tail risks across the set of K benchmark strategies:

$$\overbrace{D^*(\Pi^k \# \mathbb{P}_G) \approx D^*(\Pi^k \# \mathbb{P}_r)}^{\text{Goal of the generator}} \approx \underbrace{\Big(\mathrm{VaR}_\alpha(\Pi^k \# \mathbb{P}_r),\ \mathrm{ES}_\alpha(\Pi^k \# \mathbb{P}_r)\Big)}_{\text{Goal of the discriminator}}$$

Mathematically,

$$\min_G \frac{1}{K} \sum_{k=1}^{K} \mathbb{E}_{\mathbf{p} \sim \mathbb{P}_r} \Big[S_\alpha \Big(\underbrace{D^*(\Pi^k \# \mathbb{P}_G)}_{\text{VaR and ES prediction from } D^*} ;\ \Pi^k(\mathbf{p}) \Big) \Big]$$

Universal approximation of the generator

Theorem (Cont et al. [2025]):

Assume mild Lipschitz and regularity conditions. For any given error $\varepsilon > 0$, there exists a fully connected and feed-forward deep neural network G_1 with ReLU activation such that

$$\left|\mathrm{VaR}_\alpha\Big(\Pi^k\#((\nabla G_1)\#\mathbb{P}_z)\Big) \;-\; \mathrm{VaR}_\alpha\Big(\Pi^k\#\mathbb{P}_r\Big)\right| < \varepsilon.$$

for all $1 \le k \le K$. Here

- The depth of G_1: $L = \lceil \log n_1 \rceil$;
- The width of G_1: $N = 2^L = 2^{\lceil \log n_1 \rceil}$;
- $n_1 = \mathcal{O}(\varepsilon^{-2})$.

Universal approximation of the generator

Theorem (Cont et al. [2025])

Assume mild Lipschitz and regularity conditions ($\mathbb{P}_r$ has bounded βth moment $1 < \beta \leq 2$). For any given error $\varepsilon > 0$, there exists a fully connected and feed-forward deep neural network G_2 with ReLU activation such that

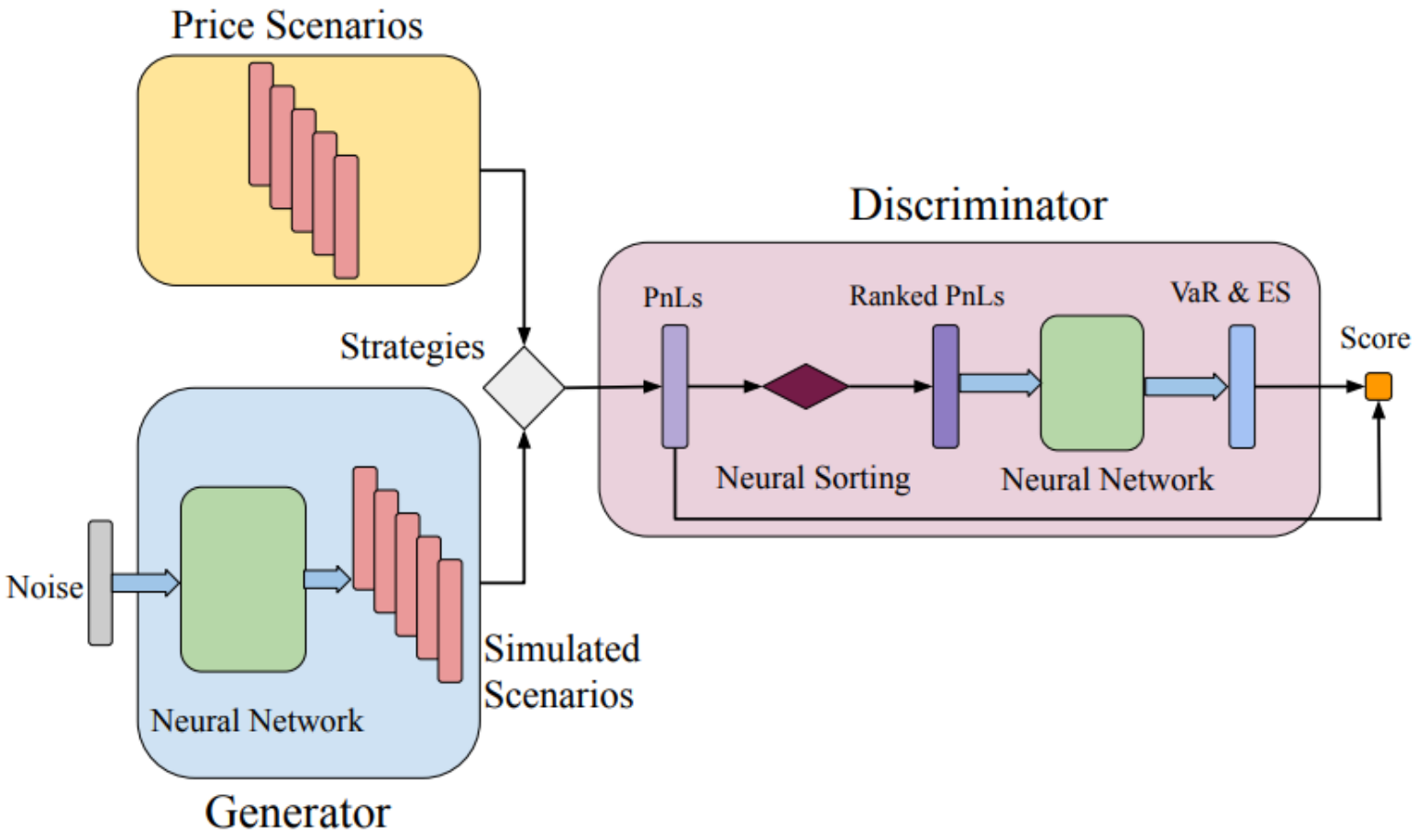
$$\left|\mathrm{ES}_\alpha\big(\Pi^k \#\big((\nabla G_2)\#\mathbb{P}_z\big)\big) - \mathrm{ES}_\alpha\big(\Pi^k \#\mathbb{P}_r\big)\right| < \varepsilon.$$

for all $1 \leq k \leq K$. Here

- *The depth of G_2: $L = \lceil \log n_2 \rceil$.*
- *The width of G_2: $N = 2^L = 2^{\lceil \log n_2 \rceil}$.*
- $n_2 = \mathcal{O}\big(\varepsilon^{-\beta/(\beta-1)}\big)$.

- Similar results also hold for risk measures that are Hölder continuous under the Wasserstein measure.

Summary of the architecture



From bi-level optimization to max–min game

Our goal is to find a generator G^* and a discriminator D^* via the following bi-level optimization problem

$$G^* \in \arg\min_G \frac{1}{K}\sum_{k=1}^{K} \mathbb{E}_{\mathbf{p}\sim\mathbb{P}_r}\Big[S_\alpha\big(D^*(\Pi^k \# \mathbb{P}_G),\ \Pi^k(\mathbf{p})\big)\Big],$$

where

$$D^* \in \arg\min_D \frac{1}{K}\sum_{k=1}^{K} \mathbb{E}_{\mathbf{p}\sim\mathbb{P}_r}\Big[S_\alpha\big(D(\Pi^k \# \mathbb{P}_r),\ \Pi^k(\mathbf{p})\big)\Big].$$

$\hookrightarrow$ constrained optimization problem is usually difficult to solve in practice!

From bi-level optimization to max–min game

Theorem (Cont et al. [2025])

Under mild conditions, the bi-level optimization problem is equivalent to the following max–min game:

$$\max_{D} \min_{G} \frac{1}{K} \sum_{k=1}^{K} \Bigg[\mathbb{E}_{\mathbf{p}\sim\mathbb{P}_r} \Big[S_\alpha\big(D(\Pi^k \# \mathbb{P}_G),\, \Pi^k(\mathbf{p})\big) \Big] - \lambda\, \mathbb{E}_{\mathbf{p}\sim\mathbb{P}_r} \Big[S_\alpha\big(D(\Pi^k \# \mathbb{P}_r),\, \Pi^k(\mathbf{p})\big) \Big] \Bigg].$$

with any Lagrange multiplier $\lambda > 0$.

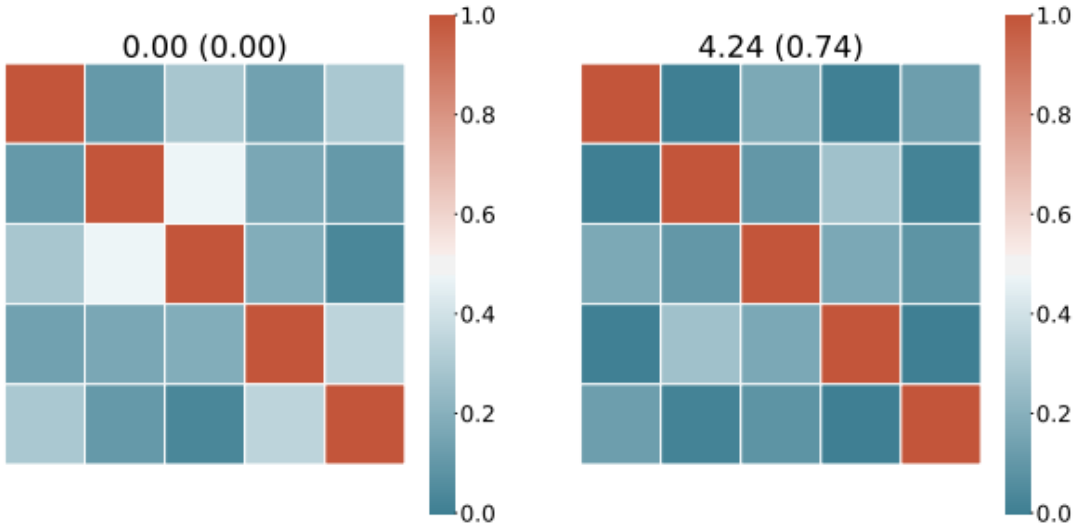
Motivation and Introduction

Score Functions for Risk Measures

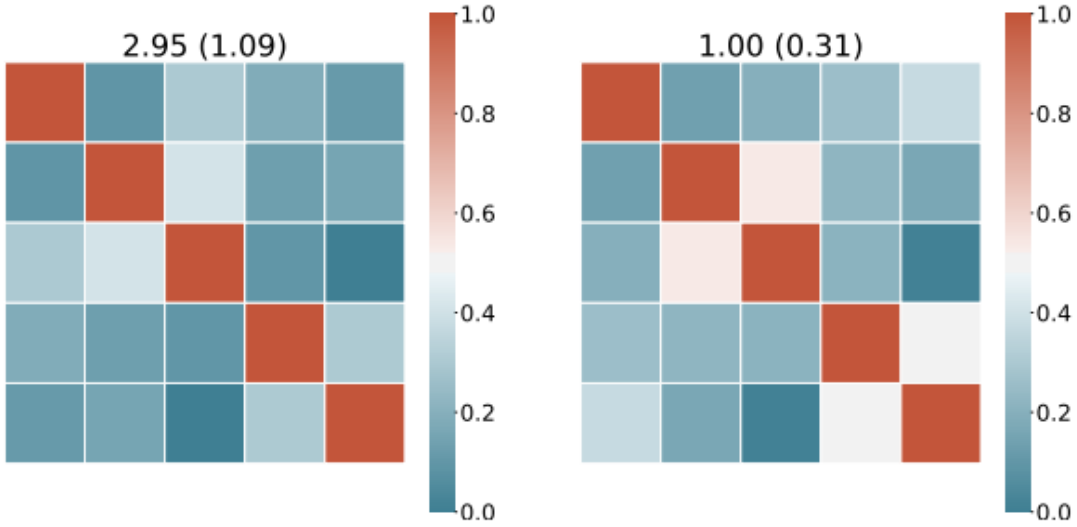
Tail-GAN: Design and Theoretical Guarantee

Numerical Performance

Correlation patterns

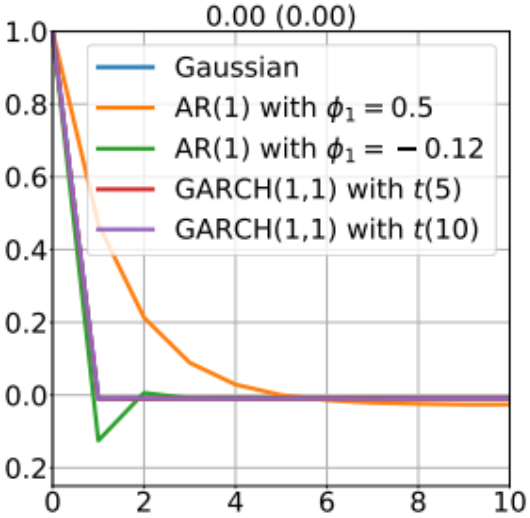


(c) Market data. (d) TAIL-GAN-Raw.

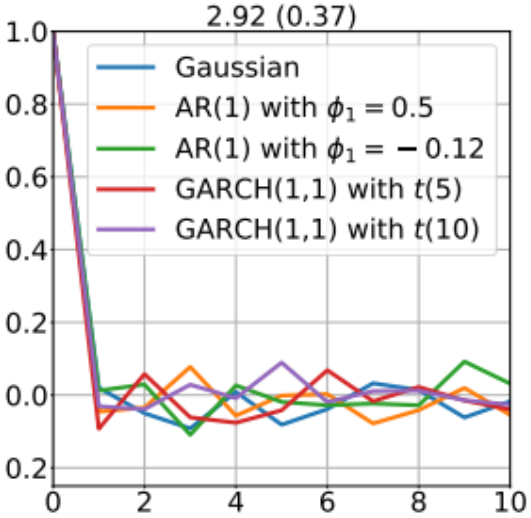


(e) TAIL-GAN-Static. (f) TAIL-GAN.

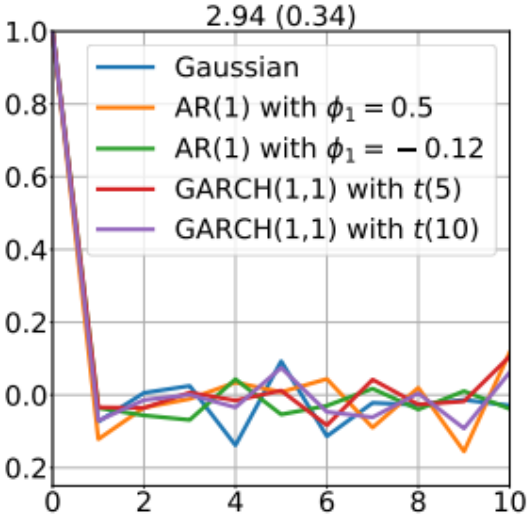
Temporal patterns



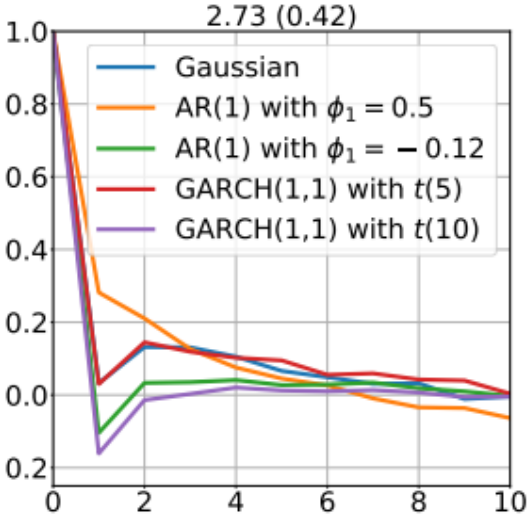
(a) Market data.



(b) TAIL-GAN-Raw.



(c) TAIL-GAN-Static.



(d) TAIL-GAN.

Performance: out-of-sample error

	"Oracle"	TAIL-GAN-Raw	TAIL-GAN-Static
%	3.0	83.3	86.7
(std)	(2.2)	(3.0)	(2.5)
	TAIL-GAN	WGAN	
	4.6	21.3	
	(1.6)	(2.2)	

Thank you!

https://pubsonline.informs.org/doi/abs/10.1287/mnsc.2023.00

Carlo Acerbi and Balázs Szekely. Back-testing expected shortfall. *Risk*, 27(11):76–81, 2014.

Hans Buehler, Blanka Horváth, Terry Lyons, Ignacio Pérez Arribas, and Ben Wood. A data-driven market simulator for small data environments. SSRN Working Paper 3632431, 2020. Available at SSRN 3632431.

Rama Cont, Mihai Cucuringu, Renyuan Xu, and Chao Zhang. Tail-gan: Learning to simulate tail risk scenarios. *Management Science*, 2025.

Tobias Fissler and Johanna F. Ziegel. Higher order elicitability and osband's principle. *Annals of Statistics*, 44(4):1680–1707, 2016.

Ian Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. Generative adversarial nets. In *Advances in Neural Information Processing Systems 27*, pages 2672–2680, 2014.

Adriano Koshiyama, Navid Firoozye, and Philip Treleaven. Generative adversarial networks for financial trading strategies fine-tuning and combination. *Quantitative Finance*, pages 1–17, 2020. Advance online publication.

Magnus Wiese, Robert Knobloch, Ralf Korn, and Patrick Kretschmer. Quant gans: Deep generation of financial time series. *Quantitative Finance*, pages 1–22, 2020. Advance online publication.

Jinsung Yoon, Daniel Jarrett, and Mihaela van der Schaar. Time-series generative adversarial networks. In *Advances in Neural Information Processing Systems 32*, pages 5508–5518, 2019.

Which score function to use?

In general, the score function is not unique...

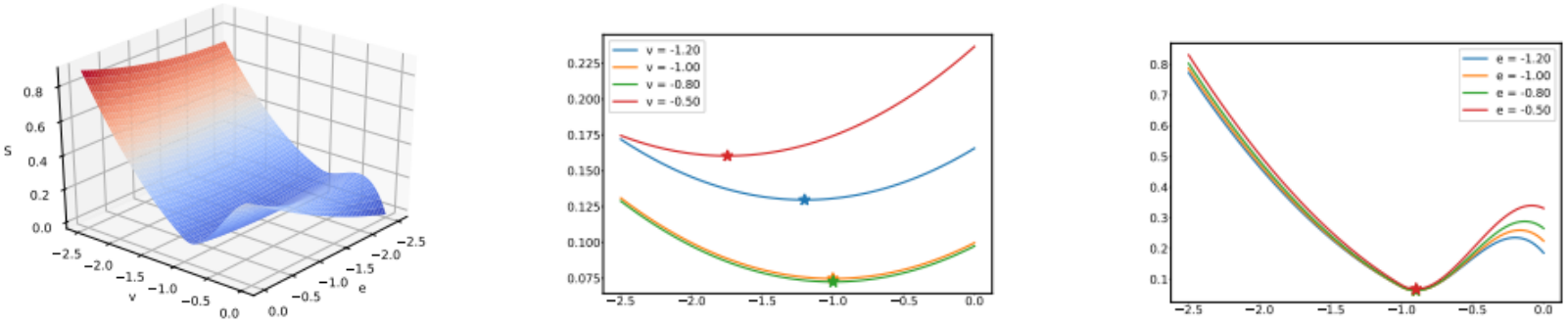
$$\begin{aligned} S_\alpha(v,e,x) &= (\mathbf{1}_{\{x\le v\}} - \alpha)\,(H_1(v) - H_1(x)) \\ &\quad + \frac{1}{\alpha} H_2'(e)\,\mathbf{1}_{\{x\le v\}}(v-x) + H_2'(e)(e-v) - H_2(e) \end{aligned}$$

- Fissler and Ziegel [2016] proposes: $H_2 : \mathbb{R} \to \mathbb{R}$ strictly convex and $R_\alpha(v,e) := \frac{1}{\alpha} v H_2'(e) + H_1(v)$ strictly increasing in v.
- Acerbi and Szekely [2014] propose: $H_1(v) = -\frac{W_\alpha}{2} v^2, \quad H_2(e) = \frac{\alpha}{2} e^2, \quad$ with $\frac{ES_\alpha(\mu)}{VaR_\alpha(\mu)} \;\ge\; W_\alpha \;\ge\; 1$

Which score function to use?

Proposition [Cont et al., 2025]

The score function proposed in Acerbi and Szekely [2014] has a better optimization landscape near the global minimum.



(a) $s_\alpha(v, e)$ (b) $s_\alpha(v, e)$ as a function of e (c) $s_\alpha(v, e)$ as a function of v

Figure 3: Landscape of $s_\alpha(v, e) = \mathbb{E}_{X \sim U([-1,1])}[S_\alpha(v, e, X)]$ based on Acerbi and Szekely [2014]